

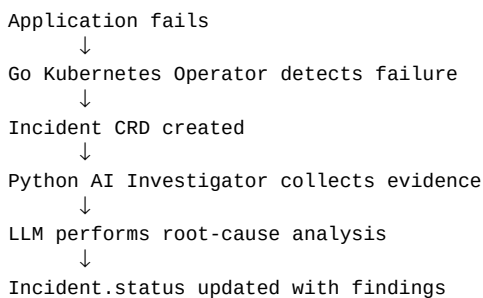
# AI-Powered Kubernetes Incident Response Platform

## End-to-End Project Plan

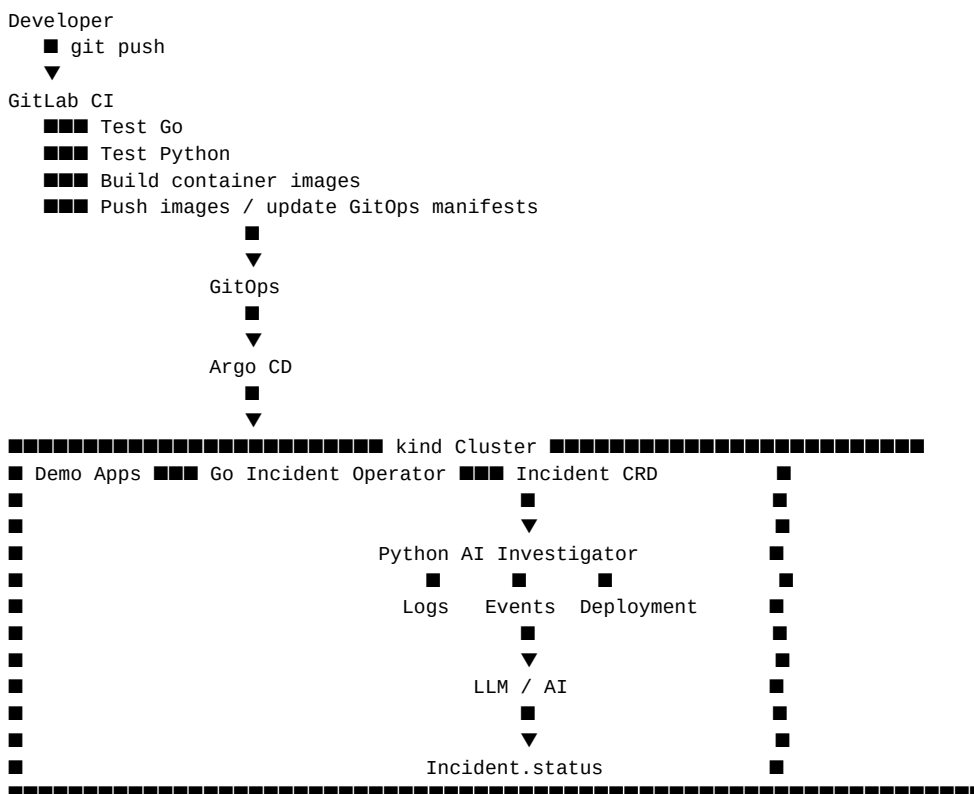
A hands-on project combining Go, Python, AI/LLMs, Kubernetes, kind, GitOps, Argo CD, and GitLab CI/CD. Designed to be built incrementally in roughly 30–40 minute daily sessions.

## 1. Project Goal

Build a platform that automatically detects Kubernetes failures, creates an Incident custom resource, investigates the incident with an AI-powered Python service, and stores the analysis back in Kubernetes.



## 2. Final Architecture



## 3. Technology Responsibilities

### Go: Kubernetes Brain

A Kubernetes operator/controller watches cluster resources and detects CrashLoopBackOff, OOMKilled, ImagePullBackOff, failed Jobs, Pending Pods, and later deployment or storage failures.

## Python: AI Brain

The Python service watches Incident resources, gathers Kubernetes context, calls the LLM, validates structured output with Pydantic, and writes the analysis back to Incident status.

## Argo CD: Deployment Engine

Argo CD continuously reconciles the desired state stored in Git with the kind cluster. CI builds and publishes artifacts; Argo CD performs deployment.

## GitLab CI: Build and Validation

GitLab CI runs tests, builds Go and Python images, pushes images, and updates GitOps configuration. The final design avoids CI directly deploying workloads into the cluster.

# 4. Incident CRD

The Incident custom resource is the contract connecting the Go operator and the Python AI service.

```
apiVersion: platform.example.com/v1alpha1
kind: Incident
metadata:
  name: payment-api-crash
  namespace: demo
spec:
  resource:
    kind: Pod
    name: payment-api-abc123
    incidentType: CrashLoopBackOff
status:
  phase: Analyzed
  analysis:
    rootCause: DATABASE_URL environment variable is missing
    evidence:
      - Application logs show DATABASE_URL not found
      - Deployment does not contain DATABASE_URL
    confidence: 94
    recommendation: Add DATABASE_URL from database-secret
    risk: Medium
```

# 5. Repository Structure

```
ai-k8s-incident-platform/
├── go-operator/
│   ├── api/v1alpha1/
│   ├── controllers/
│   ├── config/
│   ├── Dockerfile
│   ├── main.go
│   └── ai-service/
│       ├── app/
│       ├── incident/
│       ├── kubernetes/
│       ├── ai/
│       └── Dockerfile
├── demo-apps/
│   ├── healthy-app/
│   ├── crashloop-app/
│   ├── oom-app/
│   └── db-failure-app/
├── gitops/
│   ├── root/
│   ├── applications/
│   └── apps/
```

- ■■■ operator/
- ■■■ ai-service/
- ■■■ demo-apps/
- docs/
- ■■■ architecture.md
- ■■■ decisions.md
- README.md

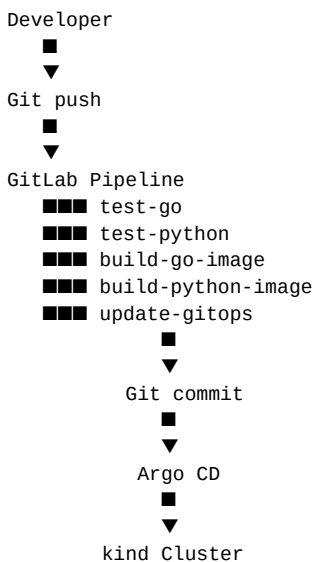
## 6. Argo CD GitOps Design

```

root-app
■
■■■ operator-app
■ ■■■ Go Incident Operator
■■■ ai-service-app
■ ■■■ Python AI Investigator
■■■ demo-apps-app
    ■■■ Healthy and intentionally broken applications
  
```

The initial implementation can use one repository for simplicity. The GitOps directory contains the manifests or Kustomize structure consumed by Argo CD.

## 7. CI/CD Flow



Start with versioned image tags if that keeps the learning flow simple. Later evolve to immutable image digests such as registry/image@sha256:... .

## 8. End-to-End Incident Flow

1. Application change is deployed.
2. The application fails, for example because DATABASE\_URL is missing.
3. The Pod enters CrashLoopBackOff.
4. The Go operator detects the failure.
5. The operator creates an Incident resource.
6. The Python service notices the new Incident.
7. Python collects pod information, logs, events, and deployment configuration.
8. The LLM analyzes the evidence.
9. The response is validated as structured output.
10. Python updates Incident.status.
11. A user runs `kubectl get incidents` or `kubectl describe incident`.

## 9. Development Roadmap: 30–40 Minutes Per Day

### Phase 0 — Foundation

**Day 1:** Create the repository, verify local tools, create the kind cluster, and install Argo CD.

**Day 2:** Create the initial GitOps structure and deploy a simple application through Argo CD. Target: Git change → Argo CD → kind.

### Phase 1 — Demo Applications

**Day 3:** Build a minimal Python HTTP application with / and /health, Dockerize it, and deploy it through GitOps.

**Day 4:** Create the first intentionally broken application by removing a required environment variable. Understand CrashLoopBackOff manually.

### Phase 2 — Go and Kubernetes

**Day 5:** Create the Go project and understand its basic structure. No controller complexity yet.

**Day 6:** Define and install the Incident CRD. Confirm kubectl can create and list Incident resources.

**Day 7:** Create a controller-runtime controller that watches Incident resources and logs reconciliation.

**Days 8–9:** Add Pod watching and detect CrashLoopBackOff. Automatically create an Incident.

**Milestone 1:** Deliberately break an app and see kubectl get incidents show the automatically created Incident.

### Phase 3 — Python Investigation Service

**Day 10:** Create the FastAPI service with health and incident endpoints and deploy it through Argo CD.

**Day 11:** Connect Python to Kubernetes and implement get\_pod(), get\_pod\_logs(), and get\_events().

**Day 12:** Watch Incident resources and print the gathered investigation context. Do not add AI yet.

### Phase 4 — AI Analysis

**Day 13:** Add LLM integration using logs and events to generate an initial root-cause analysis.

**Day 14:** Require structured JSON output containing root cause, evidence, confidence, recommendation, and risk. Validate it with Pydantic.

**Day 15:** Write the validated analysis back into Incident.status.

**Milestone 2:** Broken Pod → Go detects → Incident created → Python investigates → AI analyzes → Incident updated.

### Phase 5 — Agentic Investigation

Replace the 'send all context to the model' approach with tool calling. The AI can selectively request pod logs, events, deployments, ReplicaSets, ConfigMaps, and resource limits.

AI Agent

```
#### get_pod_logs()
#### get_pod_events()
#### get_deployment()
#### get_replicaset()
#### get_configmaps()
#### get_resource_limits()
```

### Phase 6 — Additional Failure Scenarios

1. Missing environment variable → CrashLoopBackOff
2. Memory limit too low → OOMKilled
3. Invalid image → ImagePullBackOff

4. Database unavailable → connection failure
5. Later: bad rollout → deployment unavailable

## Phase 7 — Full CI/CD

After the core application works locally, add GitLab CI to test both languages, build both images, publish them, update GitOps configuration, and let Argo CD reconcile the cluster.

## Phase 8 — Advanced Features

**Incident history:** Add PostgreSQL to store incidents, analyses, and resolutions.

**RAG:** Retrieve similar historical incidents and previous successful fixes.

**GitLab integration:** Correlate incidents with recent commits, merge requests, pipeline changes, or image changes.

**Approval-based remediation:** AI proposes a fix, a human approves it, and the change is applied through Git and Argo CD. Do not start with unrestricted automatic remediation.

## 10. Final Demonstration

```
graph TD
    A[Push application change] --> B[GitLab CI builds and tests]
    B --> C[Image published]
    C --> D[GitOps configuration updated]
    D --> E[Argo CD deploys to kind]
    E --> F[Application fails]
    F --> G[Go Operator detects CrashLoopBackOff]
    G --> H[Incident CRD created]
    H --> I[Python service investigates]
    I --> J[AI calls Kubernetes tools]
    J --> K[Root cause identified]
    K --> L[Incident.status updated]
    L --> M[Human reviews recommendation]
```

## 11. Final Learning Outcome

The completed project demonstrates Kubernetes internals, Go controllers and CRDs, Python services, LLM integration, structured outputs, agentic tool calling, Docker, kind, GitOps, Argo CD, GitLab CI/CD, observability concepts, and eventually RAG and approval-based remediation.

**Recommended build order:** Foundation → Argo CD/GitOps → broken demo app → Incident CRD → Go operator → Python investigation → AI analysis → structured output → tool calling → CI/CD → history/RAG → GitOps remediation.